

Bash – From zero to working knowledge

Prelude

Alessandro Sciarra

Z02 – Software Development Center

22 June 2026

Lecture format



{22..26} June 2026



Morning: 09:00 to 12:30
Afternoon: 14:00 to 18:00 } Interactive lectures and hands-on sessions



Wonderful location in Laubach



Dr. Alessandro Sciarra



sciarra@itp.uni-frankfurt.de



Slides are full of text for later reading! { Try to listen to me rather than reading }

Who am I?

Ancient popular wisdom

«I am not saying to be Superman, I am just saying nobody has ever seen me and Superman together in the same room.»

- Member of the Software Development Center
→ Z02 project of the CRC-TR 211 collaboration
- A used-to-be “lattice practitioner”
- Now more a Research Software Engineer (RSE)
- A dreamer who flatters himself that
 - physicists will one day properly use the IT tools they use
 - IT people will teach IT to physicists and work next to them
 - physicists will be able to focus on physics
- Not really a Bash guru, but rather a self-thought Bash enthusiast



The mind setup of this course

The mind setup of this course

- 1 **Relaxed atmosphere!**
Interactive lecture: ask, ask, ask!



The mind setup of this course

Remember: —
This is not the full story!

- 2 The amount of information given in this lecture is **huge**, I am aware of it
- 3 You will not receive all the details (I tried to pass on the important information, though)
 - ↳ be active, ask, use manuals, go beyond this lecture!

- 1 **Relaxed atmosphere!**
Interactive lecture: ask, ask, ask!



The mind setup of this course

Remember: —
This is not the full story!

- 2 The amount of information given in this lecture is **huge**, I am aware of it
- 3 You will not receive all the details (I tried to pass on the important information, though)
 - ↳ be active, ask, use manuals, go beyond this lecture!
- 4 Often it is hard (very hard) to clarify all details of and exceptions to rules
 - ↳ experience and practice will fill the gap
- 5 From rules/practical examples to abstract ideas/good practices

- 1 **Relaxed atmosphere!**
Interactive lecture: ask, ask, ask!



The mind setup of this course

Remember:
This is not the full story!

- 2 The amount of information given in this lecture is **huge**, I am aware of it
- 3 You will not receive all the details (I tried to pass on the important information, though)
↳ be active, ask, use manuals, go beyond this lecture!
- 4 Often it is hard (very hard) to clarify all details of and exceptions to rules
↳ experience and practice will fill the gap
- 5 From rules/practical examples to abstract ideas/good practices
- 6 100% Bash, no focus on portability or differences among various shell languages { I am not at that level }
↳ sh, ksh, csh, tcsh, dash, zsh, fish, ...
- 1 **Relaxed atmosphere!**
Interactive lecture: ask, ask, ask!



What is Bash ?



- Scripting/command language
- Unix shell ↔ command processor
- First release in 1989, current version v5.3 (July 2025) →  [download Bash](#)

GNU Bash

Bash is the GNU Project's shell. Bash is the Bourne Again Shell. Bash is an sh-compatible shell that incorporates useful features from the Korn shell (**ksh**) and C shell (**csh**). [...] It offers functional improvements over **sh** for both programming and interactive use. In addition, most **sh** scripts can be run by Bash without modification.

What is Bash ?



- Scripting/command language
- Unix shell ↔ command processor
- First release in 1989, current version v5.3 (July 2025) →  [download Bash](#)

GNU Bash

Bash is the GNU Project's shell. Bash is the Bourne Again Shell. Bash is an sh-compatible shell that incorporates useful features from the Korn shell (**ksh**) and C shell (**csh**). [...] It offers functional improvements over **sh** for both programming and interactive use. In addition, most **sh** scripts can be run by Bash without modification.

- A scripting language which may get hard to read → you need some “code sense”
- The effort required to a newbie is quite large → you need to practice!
- ...I am here to help you fill up this gap!

Why should I know Bash?

- It is probably the shell you use* (Bash is everywhere)
- Knowing Bash a bit more allows for better problem handling
 - ↳ e.g. delegate to Bash tasks that maybe you currently do in C/C++ or Python
- It is a scripting language very close to the (Unix) operative system
- Opportunity to make your daily work-flow more comfortable

Why should I know Bash?

- It is probably the shell you use* (Bash is everywhere)
- Knowing Bash a bit more allows for better problem handling
 - ↳ e.g. delegate to Bash tasks that maybe you currently do in C/C++ or Python
- It is a scripting language very close to the (Unix) operative system
- Opportunity to make your daily work-flow more comfortable

However,  **Bash weaknesses** are known:

- Floating point math
- Lack of data structures
- Try/catch and Exception handling: forget them!
- Functions: a tricky world!
- ...

Knowing another scripting language (e.g. Python) helps!

Why should I know Bash?

- It is probably the shell you use* (Bash is everywhere)
- Knowing Bash a bit more allows for better problem handling
 - ↳ e.g. delegate to Bash tasks that maybe you currently do in C/C++ or Python
- It is a scripting language very close to the (Unix) operative system
- Opportunity to make your daily work-flow more comfortable

Python can do everything Bash can do!?

Technically yes, just as e.g. C++ or Julia can do everything Python can do. The real question is whether you're expressing the problem at the right level of abstraction.

Bash does not compete with Python, it completes it.



What to expect from the lecture?

My point of view as physicist

The way you program today is better than it was yesterday and probably worse than it will be tomorrow! Bash has some kind of subjective corners where you will have to choose your way. Have a good reason to do things the way you do them. It will happen that you will change mind and consequently your approach to the same problem. Again and over again, it is normal. Do not hesitate in doing so. To change mind and approach means to evolve and to become a better programmer.

- This is especially important as physicist
- Knowing all details and tweaks of Bash is probably not your goal
↳ nor is it probably needed in your daily work
- **Having an overview** is, instead, very important (e.g. to **decide when to use Bash**)
- As soon as something smells odd, it is important **to be able to have a detailed look**

$\sum \int (\dots) = \int \sum (\dots)$ until something starts looking odd

References for future use

About Bash world:

- 🔗 Bash official manual
- 🔗 List of Bash builtins with descriptions
- 🔗 Notable changes in released Bash versions { More extensive list 🔗 also available }
- 🔗 A nice Bash cheatsheet
- 🔗 Google Shell Style Guide
- 🔗 A (sadly not maintained) Bash library { Clone from Github to use it, reference website is unfortunately down }
- 🔗 An online help to get relevant manual pages of a given command



Didactical material:

- 🔗 Bash Beginners Guide { A nice, from-zero tutorial }
- 🔗 Shell scripting tutorial { Nice tutorial, but not only Bash }
- 🔗 Bash Guide { Very precise guide, not completely from zero, though – AKA Greg's Wiki }
- 🔗 All material of this lecture

} Material used to prepare this lecture

↖ Main reference for preparing this course

→ My GitHub profile



What will we explore?

- 1 Inception and philosophy
- 2 Commands and arguments
- 3 Strings and types of commands
- 4 Quoting and special characters
- 5 Variables and special parameters
- 6 Shell expansion and word splitting
- 7 Patterns
- 8 Flow control
- 9 Colours and terminal commands
- 10 Namerefs and indirect expansion
- 11 Arrays and associative arrays
- 12 Input and output
- 13 Shell options and GNU Coreutils
- 14 File descriptors and redirection
- 15 Compound commands
- 16 Functions
- 17 Script autocompletion and GNU Readline
- 18 Asynchronous commands
- 19 Traps
- 20 Error handling
- 21 Sed
- 22 Awk
- 23 Some good practices and useful tools
- 24 Bash in real life
- 25 An interactive summary
- 26 What to do now?

What will we explore?

- 1 Inception and philosophy
- 2 Commands and arguments
- 3 Strings and types of commands
- 4 Quoting and special characters
- 5 Variables and special parameters
- 6 Shell expansion and word splitting
- 7 Patterns
- 8 Flow control
- 9 Colours and terminal commands
- 10 Namerefs and indirect expansion
- 11 Arrays and associative arrays
- 12 Input and output
- 13 Shell options and GNU Coreutils

Monday

22

Tuesday

23

Wednesday

24

Thursday

25

Friday

26

- 14 File descriptors and redirection
- 15 Compound commands
- 16 Functions
- 17 Script autocompletion and GNU Readline
- 18 Asynchronous commands
- 19 Traps
- 20 Error handling
- 21 Sed
- 22 Awk
- 23 Some good practices and useful tools
- 24 Bash in real life
- 25 An interactive summary
- 26 What to do now?

A slide about notation

Slides notation

- Slides are (too) full of text for a later reading!
- Many examples to immediately show you how rules apply
- In Bash code, syntax highlighted to help recognise the elements
- Prompt commands are always preceded by a dollar sign

```
$ ls
```

{ When the \$ is absent, the code refers to a script or it is pseudo-code }

Exercise colour notation

- Review the morning lecture
- Focusing on and playing with a Bash feature
- Inspirational for real life
- Technical

Clickable link
to clickable ToC

A (first) slide about AI and LLMs

A couple of questions?

- 1 How often do you chat with an AI?
- 2 How often do you use an AI in agent mode?



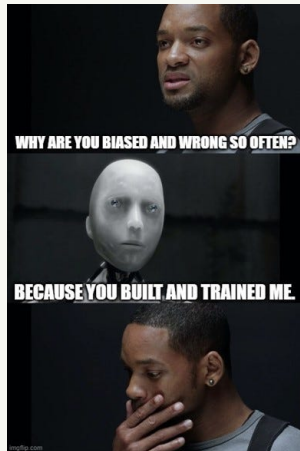
A (first) slide about AI and LLMs

A couple of questions?

- 1 How often do you chat with an AI?
- 2 How often do you use an AI in agent mode?

Let's go back few years

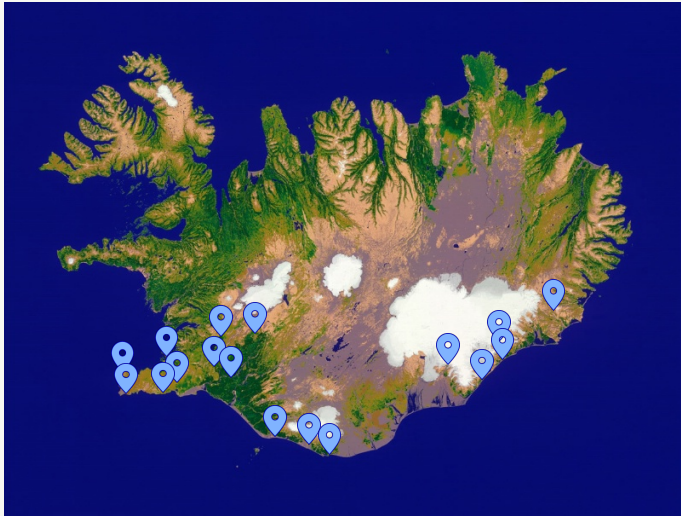
In the first two days (at least), let's avoid using any AI powered tool! Use your favourite search engine (not in AI-mode) and actively read sources.



Bonus track

Bonus track: Iceland

The magic of an incredible island



© Photos throughout the lecture: All rights are reserved

A short but intense journey

A few more recommendations

- I am aware that you will receive a lot (really a lot) of information
- Do not hesitate to ask!
- Put effort into the exercises
- The more active you are the more information will stay in your memory for your daily work

What do you expect?

A short but intense journey

A few more recommendations

- I am aware that you will receive a lot (really a lot) of information
- Do not hesitate to ask!
- Put effort into the exercises
- The more active you are the more information will stay in your memory for your daily work

What do I expect?

- You will also learn general coding principles, which are independent of the language
↳ Check out  «Clean code» and «Clean testing» talks!
- **You ask many questions!** { «There is no such thing as a dumb question.» – Carl Sagan }
- **You enjoy the journey!** { ...at least discovering Iceland... }
- **You give me feedback at the end of the lecture!**

A short but intense journey

A few more recommendations

- I am aware that you will receive a lot of information
- Do not hesitate to ask!
- Put effort into the exercises
- The more active you are, the more information you will store in your memory for your daily work

What do I expect?

- You will also learn general coding concepts that are independent of the language
↳ Check out [«Clean code»](#) by Robert Martin!
- **You ask many questions!** (e.g. { «Clean code» by Robert Martin })
- You enjoy the journey! (e.g. { «Clean code» by Robert Martin })
- You give me feedback at: **Are you ready?**